

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED
ELETTRICA E MATEMATICA APPLICATA



Mobile Robots for Critical Missions

Turtlebot4 Autonomous Navigation

Final project

Professors:

Prof. Alessia Saggese

Students:

Carmine Aliberti

Charlotte Boucherie

YEAR 2024/2025

Contents

1	Introduction	2
1.1	Project	2
1.2	Turtlebot4	3
2	Implementation	5
2.1	Architecture	5
2.1.1	Cone Detection Node	5
2.1.2	Waypoint Manager Node	6
2.1.3	Navigation and Kidnapping	6
2.1.4	Summary of Topic Flow	7
2.2	Navigation	8
2.2.1	Goal Switch	8
2.2.2	Intermediate Goals	10
2.2.3	Final Goal	10
2.2.4	Kidnapped robot	11
2.3	Cones	12
2.3.1	Cone detection	12
2.3.2	Depth Estimation and 3D Position	14
2.3.3	Cone Identification	15
2.3.4	Publishing Cone Pose Messages	16
2.4	Waypoint manager	17
2.4.1	Calculate pose of the waypoint	17
2.4.2	Transformation into the map	17
2.4.3	Buffers	18
2.5	Launcher	20
2.5.1	Building	20
2.5.2	Launching script	20
3	Problems	21
3.0.1	GPU Limitations	21
3.0.2	TurtleBot and Network Issues	22
3.0.3	Camera Availability	22
3.0.4	Limited Manpower	22
4	Conclusion	23

Chapter 1

Introduction

1.1 Project

The goal of this project is to develop an autonomous navigation system for the TurtleBot4 robot, leveraging prior knowledge of the environment provided in the form of a map done with SLAM. The system should enable the robot to move from a specified starting point, which we call the initial pose, to a target destination by defining their respective poses. The robot should also go between cones it may encounter on his path, recognize situations of kidnapping, and adapt to obstacles, wether they are static or moving.

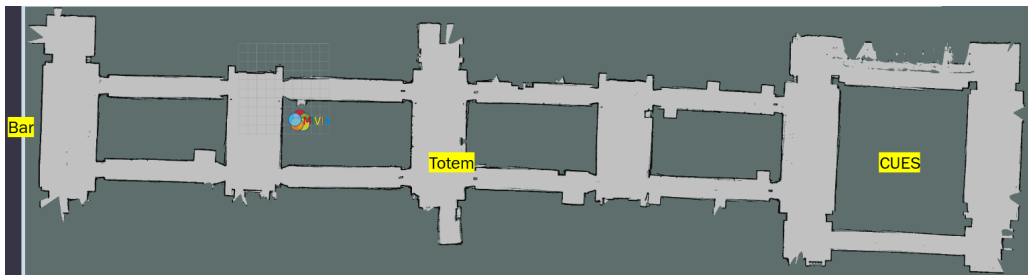


Figure 1.1: Map given for the project

During navigation, the robot must be able to:

- Plan and follow the shortest path to the destination.
- Detect, in real time, the presence of red and yellow cones placed unpredictably along the path.
- Dynamically adapt its trajectory to always pass **to the left of red cones** and **to the right of yellow cones**. This implementation in the end follows the diagram provided rather than the written instructions, as they were contradictory. The diagram was clearer and more reliable as it explains visually.
- Handle the presence of both static and dynamic obstacles, avoiding collisions.
- Handle the presence of static and dynamic obstacles, avoiding collisions.
- Operate robustly even in complex scenarios, such as closely spaced cones or sudden obstacles.

The software was developed using the ROS 2 framework, which enables modular development through its publisher-subscriber architecture based on topics. This mechanism makes

communication between nodes and message exchange easier, ensuring functional separation and improved code modularity. The system also integrates a YOLOv8 network for cone detection and tracking, allowing accurate identification of cone colors and appropriate navigation behavior in response.

1.2 Turtlebot4

The TurtleBot4 is the robotic platform used for the development of this project. It features a differential drive system with two motorized wheels, allowing it to move forward, backward, and rotate on its vertical axis. The robot can reach a maximum speed of approximately 0.31 m/s, which is suitable for indoor autonomous navigation tasks, as the task given for our project.



Figure 1.2: Turtlebot4

Equipped with a 2D LiDAR sensor, the TurtleBot4 can accurately measure distances and detect obstacles along its path, enabling real-time mapping and collision avoidance. Additionally, the onboard Inertial Measurement Unit (IMU) provides reliable estimates of the robot's orientation, contributing to stable and accurate motion control during navigation. In the end, this possibility has not been chosen and the RGB camera associated to the Stereo camera were used.

The robot is also equipped with a OAK-D Pro camera, which provides both high-quality RGB imaging and depth estimation, through its integrated stereo vision system. This sensor enables the robot to perceive its environment in 3D, making it essential for tasks such as ob-

stacle detection, object localization, and scene understanding. This camera is used for the cone detection, as the model requires to see the RGB camera to detect the cone, and the Stereo camera is used to deduce the distance according to the positions of the cone and give instructions on where to go.

In this project, the RGB camera is used at a resolution of 400×400 pixels, optimized for real-time inference with deep learning models such as YOLOv8. It was previously smaller, but in order to have better performances for the cone detection, the resolution has been set higher for this camera. The stereo camera provides depth data at a resolution of approximately 1200×720 pixels, offering a good balance between spatial accuracy and computational efficiency.



Figure 1.3: Stereo Camera

Chapter 2

Implementation

2.1 Architecture

To organize the robot's behavior efficiently and maintain a clean modular design, the overall system is split into the following functional tasks, each one representing a node:

- Cone detection
- Management of the cone
- Navigation and Kidnapping Detection

Each of these tasks is encapsulated in a dedicated ROS 2 node, except for the kidnapping detection, which is paired with navigation. Since kidnapping detection is implemented as a subscriber to a specific status topic and directly affects the robot's localization and motion behavior, it was merged with the navigation node.

To enable communication and coordination between nodes, several standard and custom ROS 2 topics are used. A custom message was implemented, as no built-in message matches the Cone directly in ROS 2 architecture. The structure of this message is detailed in the Cone Detection section.

Below is a detail of the links between our nodes in our architecture.

2.1.1 Cone Detection Node

The cone detection, first, relies on the use of the RGB Camera and the use of the Stereo Camera for the depth. It also needs the camera information, to be able to have the equivalent coordinates between the two. It subscribes to the following topics:

```
1 self.rgb_sub = self.create_subscription(  
2     Image, '/oakd/rgb/preview/image_raw', self.rgb_callback, 10)  
3 self.depth_sub = self.create_subscription(  
4     Image, '/oakd/stereo/image_raw', self.depth_callback, 10)  
5 self.camera_info_sub = self.create_subscription(  
6     CameraInfo, '/oakd/rgb/preview/camera_info', self.camera_info_callback, 10)
```

Once cones are detected, they are published on a custom message topic:

```

1 self.cones_pub = self.create_publisher(ConeArray, '/detected_cones', 10)

```

This message includes each cone's pose and color. This information serves as the primary input for the waypoint manager.

2.1.2 Waypoint Manager Node

This node processes the detected cones and decides where the robot should navigate next by computing intermediate waypoints.

To perform these tasks, the node subscribes to the list of detected cones:

```

1 self.cones_sub = self.create_subscription(
2     ConeArray, '/detected_cones', self.cones_callback, 10)

```

It also listens for feedback from the navigation node to determine whether the previous goal has been reached:

```

1 self.goal_sub = self.create_subscription(
2     Bool, '/intermediate_goal_reached', self.goal_callback, 10)

```

Once a reliable waypoint is computed and confirmed through temporal filtering, it is published as a navigation goal:

```

1 self.waypoint_pub = self.create_publisher(
2     PoseStamped, '/intermediate_goals', 10)

```

2.1.3 Navigation and Kidnapping

This node integrates navigation and re-localization logic. It receives intermediate waypoints from the waypoint manager and drives the robot toward them using a navigation stack, in our case nav2. It also monitors if the robot has been kidnapped.

Its subscriptions include:

```

1 self.create_subscription(PoseWithCovarianceStamped, '/amcl_pose',
2     self.amcl_pose_callback, qos)
3 self.create_subscription(PoseStamped, '/intermediate_goals',
4     self.intermediate_goal_callback, qos)
5 self.create_subscription(KidnapStatus, '/kidnap_status',
6     self.kidnapped_status_callback, qos)

```

It also publishes whether the robot has reached the current goal, which informs the waypoint manager that it can send the next one:

```

1 self.intermediate_goal_reached_pub = self.create_publisher(
2     Bool, '/intermediate_goal_reached', qos)

```

Finally, it initializes the localization system by setting the robot's initial position:

```

1 self.initial_pose_pub = self.create_publisher(
2   PoseWithCovarianceStamped, '/initialpose', 10)

```

2.1.4 Summary of Topic Flow

- Cone Detection → Waypoint Manager: `/detected_cones` (custom message)
- Waypoint Manager → Navigation: `/intermediate_goals` (PoseStamped)
- Navigation → Waypoint Manager: `/intermediate_goal_reached` (Bool)
- Other → Navigation: `/kidnap_status` (KidnapStatus)
- Navigation → Localization System: `/initialpose`, `/amcl_pose`

This architecture ensures each task is isolated from the others, and preserves maintainability. The principles of our algorithm are detailed in the below part.

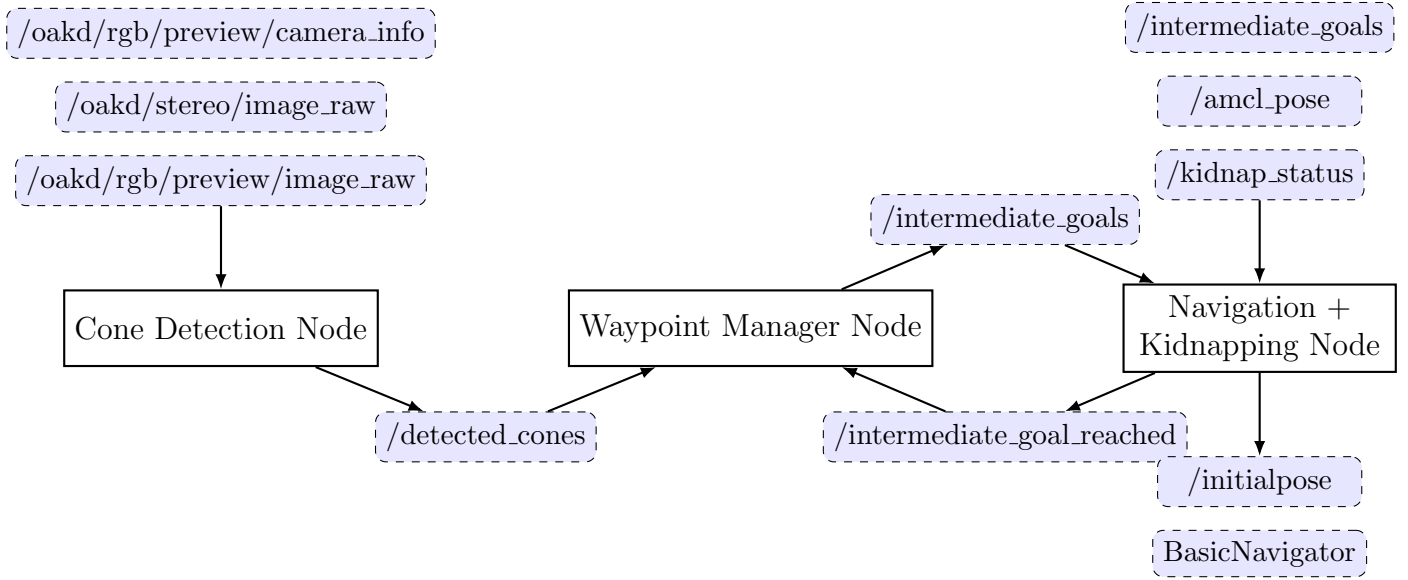


Figure 2.1: System Architecture Diagram Showing ROS 2 Nodes and Topic Communication

2.2 Navigation

The navigation logic is based on a finite state machine that defines the possible actions the robot can perform. In particular, the robot's movement is managed through three main states:

- **idle**: The initial state of the robot, representing a stationary condition with no movement.
- **intermediate**: This state corresponds to the robot moving toward an intermediate goal, represented by a pose generated based on pairs of cones or individual cones.
- **final**: The final state in which the robot moves toward the final destination, whose pose is set at the beginning of the navigation.
- **recovery**: The state used when the kidnapping get detected.

To prevent simultaneous callbacks from altering the navigation state or other shared variables, a thread locking mechanism has been implemented. This ensures that only the method which acquires the lock is allowed to modify the state-related variables, thus avoiding race conditions and ensuring safe concurrency management.

The node managing navigation is named `TryNavigator` and contains all the methods required to manage the robot's navigation from an initial point to a final destination. This node creates publishers and subscribes to the following topics:

- `/amcl_pose`: Topic subscribed to in order to obtain the TurtleBot's current pose.
- `/intermediate_goals`: Topic subscribed to in order to receive intermediate goals calculated by other nodes.
- `/intermediate_goal_reached`: Topic published to send messages signaling the successful reaching of an intermediate goal.
- `/kidnap_status`: Topic used to check if the robot is kidnapped or not.

2.2.1 Goal Switch

At startup, the robot is initialized in the final state and begins navigating toward a predefined final goal. Once this final goal is reached, the robot transitions to the idle state, indicating that no further navigation is active. The navigation node is also subscribed to the custom topic called `intermediate_goals`, which allows it to receive PoseStamped messages representing dynamically generated intermediate goals. When such a goal is received from the custom topic, the current navigation task is canceled, and the robot starts navigating toward the new intermediate destination. Upon successfully reaching the intermediate goal, the node publishes an acknowledgment message to the `intermediate_goal_reached` topic, allowing the detection node to proceed with further processing or publish new waypoints.

Based on the result obtained from `TaskResult`, the following methods are invoked accordingly:

- `handle_navigation_success()`: This method handles the successful completion of either an intermediate or the final goal. In the case of an intermediate goal, the state is set to **final** and the variable storing the last received intermediate goal is reset. For the final goal, the state is set to **idle** and a flag indicating that the final goal has been reached is enabled.

- `handle_navigation_failure()`: This method deals with the failure to reach the intermediate goal. It resets the variable holding the current intermediate goal, requests a new goal by waiting for an update on the relevant topic, and switches the state back to `final` to resume navigation toward the final destination.

The `control_loop` function was defined as follow:

```

1  def control_loop(self):
2  try:
3      self.acquire_lock("control_loop")
4
5      if self.kidnapped:
6          self.get_logger().warn("Navigazione Bloccata:
7              robot kidnapped")
8          return
9
10     # Gestione della sequenza di recovery
11     if self.state == "recovery":
12         self.perform_full_recovery()
13         return
14
15     if self.awaiting_cancel_completion:
16         if self.navigator.isTaskComplete():
17             self.get_logger().info("Cancellazione
18                 navigazione completata")
19             self.awaiting_cancel_completion = False
20             self.nav_active = False
21         else:
22
23             return
24
25     # Navigazione normale
26     if not self.nav_active and not self.final_goal_reached:
27         if self.state == "intermediate" and self.current_intermediate_goal
28         and self.intermediate_goal_start_time is not None:
29             if not self.navigate_to_intermediate_goal():
30                 self.get_logger().warn("Navigazione verso il goal intermedio fallita")
31                 elapsed_time = (self.get_clock().now() - self.intermediate_goal_start_time).na
32                 if elapsed_time > self.intermediate_goal_timeout:
33                     self.get_logger().warn(f"Timeout di
34                         {self.intermediate_goal_timeout} secondi raggiunto per il goal intermedio!")
35                     self.navigator.cancelTask()
36                     self.handle_navigation_failure() # Gestisci il fallimento
37                     self.intermediate_goal_start_time = None
38                     # Resetta il timer
39                     return
40
41
42         elif self.state == "final":
43             if not self.navigate_to_final_goal():
44                 self.get_logger().warn("Failed
45                     to start final navigation")
46
47     if self.nav_active and self.navigator.isTaskComplete():
48         result = self.navigator.getResult()
49         if result == TaskResult.SUCCEEDED:
50             self.handle_navigation_success()
51         else:
52             self.handle_navigation_failure()

```

```

53
54 finally:
55     self.release_lock("control_loop")

```

2.2.2 Intermediate Goals

As mentioned previously, intermediate goals are published on the custom topic `/intermediate_goals`, which triggers the callback method `intermediate_goal_callback`. This callback encapsulates the entire logic for managing and validating received intermediate goals. Upon receiving a new goal, a thread lock is acquired to ensure thread safety, and the `Pose` is extracted from the message. The method then checks whether the `frame_id` is correct; if not, the pose is transformed into the `map` frame using TF2.

Once the pose is in the correct frame, two critical checks are performed to ensure robust goal selection:

1. The new pose must be sufficiently different from previously received poses, to avoid executing nearly identical waypoints caused by noisy or unstable detections;
2. The new pose must not be significantly farther from the robot's current position, in order to prevent skipping closer and more relevant waypoints.
3. The new pose shouldn't have been seen before in a list of saved waypoints.

This filtering helps reduce redundant movements and improve the efficiency of the trajectory. If the intermediate goal passes both checks, the navigation state is set to `intermediate`, enabling the method `navigate_to_intermediate_goal()`. This method verifies that the robot's current pose is known, and then calculates the orientation difference between the intermediate goal and the final goal. The orientation of the intermediate pose is adjusted so that the robot remains aligned toward the final goal. This allows the robot to follow straighter paths and avoid unnecessary rotations.

After adjusting the orientation, the modified pose is sent to the `BasicNavigator` via the command `self.navigator.goToPose(goal)`. Upon successfully reaching the intermediate goal, a message of type `Bool` is published on the topic `/intermediate_goal_reached` to signal the detection node that the robot is ready to receive the next waypoint.

Finally, to prevent the system from accepting new intermediate goals after the final destination has been reached, a state variable named `final_goal_reached` is used. When this variable is set to `True`, it disables further processing of intermediate goals.

2.2.3 Final Goal

At the start of navigation, the system enters the `final`, which activates the `navigate_to_final_goal()` function. This function retrieves the current position of the TurtleBot via `amcl_pose`, calculates the positional and angular difference with respect to the final goal, and then sets this goal in the `BasicNavigator` using the `goToPose()` function. At the same time, the `nav_active` variable is set to `True` to indicate that navigation is in progress.

During this phase, if one or more intermediate goals are received on the `/intermediate_goals` topic, the robot interrupts navigation to the final goal by calling `cancelTask()`, switches the

state to **intermediate**, and updates the destination with the new intermediate goal (if valid). The intermediate goal is accepted only if it is sufficiently distant (spatially and/or angularly) from the previous one, and if it is not too far (more than 3 meters from the current position). If no new intermediate goals are received, the TurtleBot continues navigating toward the final goal.

2.2.4 Kidnapped robot

According to the project instructions, the system implements a kidnapping detection mechanism, which mechanism monitors whether the robot has been lifted or moved unexpectedly. Detection is based on a custom message type published on the topic `/kidnap_status`, specifically using the `KidnapStatus` message from the `irobot_create_msgs` package.

The navigation node `TryNavigator` subscribes to the `/kidnap_status` topic to monitor these events in real time. The callback function `kidnapped_status_callback` updates the internal `kidnapped` state variable, which is used in the main control loop to disable navigation during kidnapping scenarios.

When a kidnapping event is detected, the following steps are executed inside the `handle_kidnapping()` method:

1. **Navigation Interruption:** Any active navigation task is immediately canceled using `cancelTask()`, and the `nav_active` flag is reset to **False**. This prevents the robot from attempting to continue its path based on potentially invalid pose estimates. The robot state is then set to **recovery**.
2. **Pose Recovery:** The robot attempts to restore its most recent known valid pose saved before. The robot then rotate and tries to locate itself in the environment.
3. **State Reset:** The robot's navigation state is reverted to **final**, any stored intermediate goal is discarded, and the flag `final_goal_reached` is set to **False**. Any waypoint is therefore cancelled, as the cones are not there anymore, and the robot has to continue his way towards the final goal.

Once the `kidnapped_status_callback` receives a message indicating the kidnapping has ended, that is the message stating the status is false, the internal `kidnapped` flag is reset, and a log message confirms that the robot is ready to resume normal operation. All the logic that handles the rotation is described by the function `perform_recovery_rotation`, which create a series of waypoint which differs only in orientation to make the robot rotate. The logic of setting the position and checking the tasks is done by `perform_full_recovery`. The navigation logic embedded in the `control_loop` automatically prevents further movement while `kidnapped` is **True**, in order to ensure normal behaviour during unexpected displacements.

2.3 Cones

2.3.1 Cone detection

The logic of our cone detection system is structured in two primary stages: cone detection and color classification. The system first identifies the presence and location of cones within an image using a pretrained YOLOv8 model, and then classifies the color of each detected cone through HSV (Hue, Saturation, Value) based analysis.

Model

The detection task is handled using a pretrained YOLOv8 model, sourced from the GitHub repository Safety Cone Detection, which was recommended in the project materials. It is designed to address the need for real-time detection of safety cones. It utilizes the YOLOv8 model for high accuracy and real-time performance. It offers fast frame-per-second (FPS) rates, ensuring swift detection and response. It provides robust detection capabilities, identifying safety cones across various environments.

```
1 self.model = YOLO("Cone.pt")
2 results = self.model.predict(source=frame,
3                               conf=0.6, verbose=False, stream=True)
```

The model processes the image frame captured from the RGB camera and returns detection results. Each result contains bounding box coordinates corresponding to detected cones. The confidence threshold is adjustable. In this case, a conservative threshold of 0.6 was selected to minimize false positives, such as brightly colored objects as people or doors that might otherwise be incorrectly classified as cones. This choice therefore prioritizes navigation reliability by avoiding potential misplacement of waypoints.

Color detection

After detecting potential cones in the image, each bounding box is cropped from the image. These crops are then passed through a custom color detection function to classify the dominant color of each cone.

Images from the camera are initially in the BGR color space which is the default format in OpenCV, used by our model for the detection. To improve the robustness of color detection, especially under varying lighting conditions, the crop is converted to the HSV (Hue, Saturation, Value) color space. This makes it easier to set thresholds like "detect anything reddish" without being confused by shadows or lighting, since the Hue parameter directly differentiate between the colors.

It is important to note that OpenCV uses a hue range of $[0, 180]$, whereas most color pickers use $[0, 360]$. Similarly, Saturation and Value range from 0 to 255 in OpenCV, differing from the $[0, 100]$ scale commonly used online.

```
1 def get_dominant_color(self, crop):
2     hsv = cv2.cvtColor(crop, cv2.COLOR_BGR2HSV)
3     colors = {
4         'red': [
5             ((0, 80, 50), (5, 255, 255)),
6             ((161, 80, 50), (180, 255, 255))
```

```

7         ],
8         'orange': [((6, 80, 50), (20, 255, 255))],
9         'yellow': [((21, 80, 50), (40, 255, 255))],
10        'green': [((41, 70, 50), (85, 255, 255))],
11        'blue': [((86, 70, 50), (130, 255, 255))]
12    }
13    max_color = 'undefined'
14    max_pixels = 0
15    for color_name, ranges in colors.items():
16        mask_total = None
17        for lower, upper in ranges:
18            lower_np = np.array(lower)
19            upper_np = np.array(upper)
20            mask = cv2.inRange(hsv, lower_np, upper_np)
21            if mask_total is None:
22                mask_total = mask
23            else:
24                mask_total = cv2.bitwise_or(mask_total, mask)
25        count = cv2.countNonZero(mask_total)
26        if count > max_pixels:
27            max_pixels = count
28            max_color = color_name
29    return max_color

```

The algorithm defines RGB ranges for the colors red, orange, yellow, green, and blue. For the project, only orange and yellow are supposed to be considered, but as the cones provided were of these colors, all colours possible were taken into account, to account for the limited numbers of cones given for the tests. For each color, it computes a binary mask and counts the number of non-zero pixels. The color with the highest pixel count is selected as the dominant one. If no color surpasses a minimal threshold (10 pixels), the result is marked as **undefined**.

The input of the function is a crop, and it returns a color for the classification. The reason behind the crop is to not take into account the color of the background. We take a center crop of the bounding box of the cone to keep the center, presumably where we can find the better the color of the cone.

```

1    for result in results:
2        boxes = result.bboxes.xyxy.cpu().numpy()
3        for box in boxes:
4            x1, y1, x2, y2 = box.astype(int)
5            cv2.rectangle(frame, (x1, y1), (x2, y2),
6                           (0, 255, 0), 2)
7
8            w = x2 - x1
9            h = y2 - y1
10           cx1 = max(x1 + w // 4, 0)
11           cy1 = max(y1 + h // 4, 0)
12           cx2 = min(x2 - w // 4, frame.shape[1] - 1)
13           cy2 = min(y2 - h // 4, frame.shape[0] - 1)
14
15           center_crop = frame[cy1:cy2, cx1:cx2]
16           if center_crop.size == 0:
17               continue
18
19           color_label = self.get_dominant_color(center_crop)

```

This color label is then associated with the corresponding detection, allowing us to categorize each cone as from the orange group or the yellow group. The green and blue cones given are therefore associated to the yellow cones and red cones are associated to the orange one. The chosen behaviour for the robot is to go on the right of the yellowish cones, and on the left of the orangish cones.

Overall, the performance of the color detection system is good. However, some difficulties under very bright lighting conditions were noticed. In such cases, the colors sometimes fall outside the defined thresholds and are labeled as undefined. To address this, the saturation (S) threshold was adjusted in our HSV ranges to accept lighter/ more desaturated colors. While these pale colors may not strictly fall within the ideal range of our color definitions, this adjustment works well for our specific task. Since our classification only distinguishes between color groups (e.g., orange vs. yellow), slight variations in brightness or saturation do not impact correctness, as the hue (H) value typically remains stable even under pale lighting conditions.

2.3.2 Depth Estimation and 3D Position

After the cone detection and color classifier, we have the coordinates on the cones and its color located on the RGB image from the camera. However, this image is flat and we need to get the distance to be able to locate where the waypoint should be later so that the robot knows how far it shall go. The next step is therefore to estimate its 3D position using the stereo camera's depth image. This process converts 2D image coordinates into 3D world coordinates.

It is important to note that for a cone, the position of the bounding box on the image is given thanks to the model, but it is therefore a rectangle. The coordinates used to represent our cone therefore for the next steps are the middle point in x, therefore the middle on the horizontal plane, and the bottom point in the vertical place, that is y2. The estimated distance should therefore be better, and the robot should look for the ground doing his computation to check for a point to go.

To begin, the 2D pixel coordinates from the RGB image are scaled to match the depth image resolution. This is necessary because the RGB and depth images have different resolutions, and accurate scaling ensures pixel alignment:

```

1 scale_x = depth_image.shape[1] / rgb_image.shape[1]
2 scale_y = depth_image.shape[0] / rgb_image.shape[0]
3
4 u_depth = int(x * scale_x)
5 v_depth = int(y * scale_y)
```

Then, a small window (5x5) centered around the pixel is used to collect a patch of depth values. From this patch, the median is selected to reduce the impact of noise, which is often present in stereo depth images. A decline in performance of the distance is noted when this window is not used, as it would take the point at only one instant and could be flawed, while the risk is reduce averaging it.

```

1 window_x = 5
2 window_y = 2
3
4 x_min = max(0, x_px - window_x)
5 x_max = min(w - 1, x_px + window_x)
```

```

6         y_min = max(0, y_px - window_y)
7         y_max = min(h - 1, y_px + window_y)
8
9         patch = depth_img[y_min:y_max+1, x_min:x_max+1]
10
11         valid_patch = patch[(patch > 0.1) & (patch < 10.0)]
12         if valid_patch.size == 0:
13             return None
14
15         Z = float(np.median(valid_patch))
16         if Z <= 0.3 or Z > 6:
17             return None

```

Only valid depth values are considered for the distance. Cones that are too close (≤ 0.3 m) or too far (≥ 6 m) are discarded. Occasionally, our model gave weird distance even while using the window, so it prevents the robot to take into account weird distance. The robot should therefore go to wrong waypoints, and even if these points were just, it allows the robot to progress slower on its path and not too abruptly.

Once the depth is obtained, the 3D position is computed using the camera intrinsic parameters. Since the depth image is lower resolution than the RGB image, the intrinsics are scaled accordingly.

```

1 scaled_fx = self.rgb_camera_info.k[0] * scale_x
2 scaled_fy = self.rgb_camera_info.k[4] * scale_y
3 scaled_cx = self.rgb_camera_info.k[2] * scale_x
4 scaled_cy = self.rgb_camera_info.k[5] * scale_y
5
6 x_cam = (u_depth - scaled_cx) * depth / scaled_fx
7 y_cam = (v_depth - scaled_cy) * depth / scaled_fy
8 z_cam = depth

```

The result is a point in the camera optical frame (x_{cam} , y_{cam} , z_{cam}), which is then packaged into a ROS 2 `PoseStamped` message. It is however important to note that these coordinates are still relative to the robot, and framed into the optical camera. We did the projection for the 2D plane to the 3D plane, receiving the depth, but we did not add the position of the robot, therefore not locating the cone on the map but gave the position relative to the robot.

2.3.3 Cone Identification

Each detected cone is processed individually by the detection. For each bounding box, the center point is calculated and used to estimate the 3D position of the cone using the depth image and camera intrinsics, and the color of the cone is determined. Finally, the detected cone's position and color are packed into a message of type `Cone`, which was created for our particular project, which includes a `geometry_msgs/Pose` and a `string` color. These cones are grouped together into a `ConeArray` message, as shown below.

```

1 detections = ConeArray()
2 detections.header.stamp = self.get_clock().now().to_msg()
3 for result in results:
4     boxes = result.bboxes.xyxy.cpu().numpy()
5     for box in boxes:

```



```

6         x1, y1, x2, y2 = box.astype(int)
7         center_x = (x1 + x2) // 2
8         center_y = (y1 + y2) // 2
9
10        color = self.get_dominant_color(self.latest_rgb[y1:y2, x1:x2])
11        pose = self.calculate_cone_pose(center_x, center_y)
12        if pose is None:
13            continue
14
15        cone = Cone()
16        cone.color = color
17        cone.pose = pose.pose # pose is PoseStamped
18        detections.cones.append(cone)

```

This block ensures that for every valid detection, we create a fully populated **Cone** message and append it to the array for publication.

2.3.4 Publishing Cone Pose Messages

Once all cones in a frame are processed, the complete **ConeArray** message is published on the ROS topic `/detected_cones`. This message serves as a centralized source of structured cone information. This array of **Cone** is then published, and recuperated by the waypoint managing node, which will create the waypoint for the navigation to go as an intermediate goal.

The custom messages are defined in the `my_msg_pkg` package as follows.

Cone.msg

```

1 string color
2 geometry_msgs/Pose pose

```

ConeArray.msg

```

1 Cone[] cones
2 std_msgs/Header header

```

The header timestamp is taken from the current time at the moment of publication, and all cone poses are expressed in the `oakd_rgb_camera_optical_frame`, which can be transformed later, as the waypoint needs a point on the map. Given this array of cone, our waypoint managing node is able to take decisions on the waypoint to give or not to the navigation. This way, our code is well structured and segmented.

```

1 msg = Cone()
2 msg.pose = transformed_pose.pose
3 msg.color = color_label
4 msg.header.frame_id = "map"
5 msg.header.stamp = node.get_clock().now().to_msg()
6 cone_publisher.publish(msg)

```

Above is detailed the message published and all the details on it.

2.4 Waypoint manager

Once cones are detected, classified by color, and their 3D poses estimated, the robot must interpret these spatial cues to determine intermediate navigation goals. The waypoint manager handles this process, generating and publishing intermediate waypoints that guide the robot through the cone layout.

2.4.1 Calculate pose of the waypoint

The waypoint manager groups detected cones into two categories, according to the diagram given on the slides.

- Yellow : Left-side markers.
- Orange : Right-side markers.

Using these groups, waypoints are calculated using the following rules :

- Paired Cones : The waypoint is placed midway between the cones.
- Single Cone : The waypoint is placed with a relative offset to a side of the cone, according to its colour.

The algorithm is given in the function `calculate_waypoint_on_camera`. This function takes as input the cones, with their colour and position of type `Pose()` and relative to the `camera_frame`, and gives as output a position of the type `Pose()` too.

This ensures that the robot has instruction in every case it detects a cone, and to prevent mistakes on the detection, if for example it detects only one cone instead of the two.

2.4.2 Transformation into the map

All detected cone positions are originally in the camera frame. These must be transformed into the map frame to ensure consistency with the global navigation system. The 3D cone pose estimated in the camera frame must be transformed into the map frame to be used for navigation and waypoint generation.

This is achieved using the ROS 2 TF2 library. The transformation is attempted using the transform buffer and listener initialized in the node. If the transform is not available, the function catches the exception and skips processing for that detection.

```
1 def transform_pose_to_map(self, pose_non_stamped):
2     try:
3         if not self.tf_buffer.can_transform("map", "oakd_rgb_camera_optical_frame",
4             rclpy.time.Time(), timeout=rclpy.duration.Duration(seconds=1.0)):
5             self.get_logger().warn("Transform oakd_rgb_camera_optical_frame
6                 -> map not available yet")
7             return None
8
9         transform = self.tf_buffer.lookup_transform(
10             "map", "oakd_rgb_camera_optical_frame", rclpy.time.Time(),
11             timeout=rclpy.duration.Duration(seconds=1.0)
12         )
13
```

```

14         transformed_pose = tf2_geometry_msgs.do_transform_pose(pose_non_stamped,
15                         transform)
16
17         new_pose_stamped = PoseStamped()
18         new_pose_stamped.header.frame_id = "map"
19         new_pose_stamped.header.stamp = self.get_clock().now().to_msg()
20         new_pose_stamped.pose = transformed_pose
21
22         return new_pose_stamped
23
24     except (tf2_ros.LookupException, tf2_ros.ConnectivityException,
25           tf2_ros.ExtrapolationException):
26         self.get_logger().warn("TF transform failed")
27         return None

```

2.4.3 Buffers

To avoid publishing noisy or flickering waypoints due to transient cone detections, a temporal buffer is used. When a potential waypoint is calculated, it is added to a buffer with a timestamp. Waypoints are only published when a consistent detection is confirmed over multiple callbacks.

The strategy is chosen as below :

- Waypoints are stored with their ID and timestamp.
- The buffer is filtered to retain only recent entries.
- Waypoints with the highest frequency are selected for publishing.
- The position is averaged from repeated observations to smooth noise.
- Once published, new waypoints are withheld until the robot reports reaching the intermediate goal, via the topic `/intermediate_goal_reached`).

The function verifying the qualities of the waypoints is `check_and_publish(self)`. This function is responsible for verifying the consistency and reliability of detected waypoints before publishing them for navigation. It acts as a filtering that ensures only high-confidence waypoints are passed on to the robot. However, the navigation node also does some part of this work, as it has access to the position of the robot more easily, and if the waypoint is too far to the robot, will refuse a waypoint.

```

1  def check_and_publish(self):
2      with self.buffer_lock:
3          if len(self.waypoint_buffer) < self.min_detections:
4              return
5
6          groups = defaultdict(list)
7          for wp in self.waypoint_buffer:
8              groups[wp['id']].append(wp)
9
10         best_id, best_count = None, 0
11         for wp_id, detections in groups.items():
12             if len(detections) > best_count:
13                 best_count = len(detections)

```

```

14         best_id = wp_id
15
16     if best_id is not None and best_count > 0:
17         detections = groups[best_id]
18         avg_x = sum(d['pose'].pose.position.x for d in detections)
19                 / len(detections)
20         avg_y = sum(d['pose'].pose.position.y for d in detections)
21                 / len(detections)
22         avg_angle = sum(d['angle'] for d in detections)
23                 / len(detections)
24
25         if self.is_waypoint_published(best_id, (avg_x, avg_y), avg_angle):
26             self.get_logger().info(f"Skipping duplicate waypoint")
27             return
28
29         pose = PoseStamped()
30         pose.header.frame_id = 'map'
31         pose.header.stamp = self.get_clock().now().to_msg()
32         pose.pose.position.x = avg_x
33         pose.pose.position.y = avg_y
34         pose.pose.orientation.w = 1.0
35
36         self.get_logger().info(f"Published intermediate waypoint map position
37                                id: {best_id}: X={avg_x:.2f}, Y={avg_y:.2f}, Angle={avg_angle:.2f}")
38
39         self.published_waypoints[best_id] = {
40             'x': avg_x,
41             'y': avg_y,
42             'angle': avg_angle,
43             'timestamp': time.time()
44         }
45
46         self.waypoint_pub.publish(pose)
47
48         self.can_publish = False
49         self.first_goal_published = True
50         self.waypoint_buffer.clear()

```

The function only proceeds if enough waypoint detections have been accumulated in the internal `waypoint_buffer`. This threshold (`min_detections`) ensures statistical significance. Waypoints are grouped by their assigned ID, assuming each detection has a unique or consistent identifier across frames, which is done earlier by assigning an ID according to the position on the map. Among all grouped IDs, the one with the highest number of detections is selected. This assumes the most commonly detected waypoint is likely the most reliable. The function computes the mean position and orientation angle of all detections for the selected waypoint ID to reduce the effect of measurement noise. Before publishing, it checks whether a similar waypoint has already been published, avoiding redundant data to reduce the work for the next node. It builds a `PoseStamped` message to publish to the rest of the navigation system. The frame is 'map', indicating that coordinates are global.

Once the robot reaches the current waypoint, the manager resumes publishing the next available waypoint, given the new `ConeArray` given by the cone detection node.

2.5 Launcher

The final thing implemented for this project was both script `build.sh` and `launcher.sh`, in order to not have to run every command every time we have to create a test.

2.5.1 Building

The script `build.sh` is responsible for compiling the entire ROS 2 workspace. It begins by sourcing the base ROS 2 environment from `/opt/ros/humble/setup.bash` to ensure all core tools and dependencies are available. It then runs `colcon build` in the root of the workspace, which compiles all packages and generates the appropriate build, install, and log directories. This step is required before launching any nodes, as it ensures all custom packages are correctly built and ready to be executed, and is the reason behind the choice of creating two different files.

2.5.2 Launching script

The script `launcher.sh` automates the startup of all ROS 2 nodes required for the project by opening separate terminal tabs for each component. Each tab begins by sourcing both the global ROS 2 setup and the local workspace's `install/setup.bash` to access all packages, which is required in order to have a proper functioning.

The script launches the localization stack with the map file, brings up the Nav2 navigation stack, opens the RViz visualization tool, and runs the three core nodes: `cone_detector`, `waypoint_manager`, and `navigation`. By running these commands in parallel, it eliminates the need to manually launch each node, improving testing efficiency.

Chapter 3

Problems

In this chapter are detailed all problems encountered during the development of the project and the potential solutions used to solved them.

3.0.1 GPU Limitations

One of the main challenges encountered during development was the absence of an Nvidia GPU, which significantly limited the system's ability to process camera frames from the TurtleBot. As a result, both the detection and tracking of cones were negatively affected, leading to issues in generating intermediate navigation goals. It was common for the computer to crash while testing, which added to the difficulties.

Since the system had to run entirely on CPU, several optimizations were explored to mitigate this limitation.

- **Reducing the temporal window and buffer size:** Lowering the required temporal window for detection and reducing the frame buffer size allowed for quicker goal publication and reduced latency, making the system less sensitive to processing overhead.
- **Using `stream=True` during inference:** Enabling streaming mode during predictions helped improve performance in real-time scenarios, such as live camera input, by optimizing memory and resource usage during inference. However, this problem did not really got resolved on one of the computer, since this parameter is supposedly helping only in case of a weak GPU, and not in the absence of one.
- **Increasing the confidence threshold:** A higher confidence threshold was used to reduce false positives, preventing the system from wasting computational resources on invalid or inaccurate detections. This also helped the performance indirectly, but it helped on that matter too.
- **Removing the visualization of the camera:** In the early stage of the robot, it was useful to have the visualization of where was the checkpoint position located on the RGB visualization of OpenCV. However, the computations added were not worth it in the end, when the robot needed more computational power in real time. The camera was often lagging behind, by up to five seconds.

While these patches improved the overall functionality, the lack of GPU support remained a significant problem. The detection system performs reliably in most scenarios, and the TurtleBot is generally able to complete navigation tasks, although occasional failures and inaccuracies still occur due to CPU-related constraints.

3.0.2 TurtleBot and Network Issues

The use of the TurtleBot introduced several hardware-related challenges. One of the major problems was frequent overheating, which often led to system crashes requiring manual reboot. Additionally, battery limitations severely constrained testing time. Even short experiments consumed a significant portion of the battery, while a full recharge required several hours. A low battery level also affected system performance, notably making the cameras less stable and less reliable during operation.

Moreover, network-related issues were common throughout development. Unstable Wi-Fi connections occasionally disrupted communication between the robot and remote nodes, impacting both control and data transfer, especially during real-time operations.

3.0.3 Camera Availability

The camera system on the TurtleBot was another critical point of failure. Its availability was not only dependent on the robot's battery level but was also significantly affected by the absence of GPU acceleration. In many cases, the navigation node would begin operation before the camera was fully initialized, resulting in delayed detections and consequently delayed publication of intermediate goals. This introduced additional latency and reduced the overall efficiency of navigation.

To adapt to this problem, a first idea is to make the robot does longer test, and try the cones further in the way. However, it is directly opposed to the problem about the battery of the robot, as doing that the number of tests were much more limited.

3.0.4 Limited Manpower

As the title suggests, the project was carried out by only two people. While the workload was distributed as evenly as possible, the limited number of contributors posed a considerable challenge given the time and complexity of the project. The lack of additional support, both in terms of idea exchange and time for work significantly impacted the pace of development and the depth of possible exploration and testing. The other groups were from three to four person per group.

Chapter 4

Conclusion

In this project, we designed and implemented a multi-node robotic system capable of detecting cones, generating waypoints, and navigating through an environment while handling complex scenarios such as robot kidnapping. The work was divided into the following modules: cone detection, waypoint management, and navigation with kidnapping awareness. This modular approach facilitated development, debugging, and integration.

The different nodes communicate via custom and standard messages. The cone detection node effectively processed RGB and depth images to locate and classify cones in real time. The waypoint manager used this information to generate meaningful navigation goals by interpreting the spatial relationships between cones. Finally, the navigation node ensured that the robot reached these goals while being resilient to re-localization events caused by simulated kidnapping.

Throughout the project, we faced and overcame technical challenges, such as coordinate frame transformations, synchronization of sensor data, the design of a consistent and reliable communication architecture and the limited manpower of our team, which may be one of the biggest difficulties, since of the impossibility of really overcoming it.

Ultimately, our robot can do the task of navigating from one point to another, while following cones. However, the perception-based waypoint strategy is efficient but lacks velocity and our robot suffers from being too slow. It sometimes sends the waypoint while having already passed the cone, causing new issues while trying to go backward, which we could not always resolve due to GPU limitations. The range of the camera being limited, the closeness of the cones are also a limiting point of our robot. The logic of sending the position with an offset in the case of not seeing both cones also helps a lot, as it will go between the cones and do the position correctly even if one of the cone is not detected. Sometimes, the robot will not be able to find back its position after the kidnapping, particularly if the new position is too far from the first one.